

脆弱性情報アップストリームの不安定性の実測： 履歴管理基盤上の公開前検査による 82 日間の全数運用調査

篠原 俊一^{1,a)} 中岡 典弘^{1,b)} 神戸 康多^{1,c)}

概要：脆弱性データベースは多数の上流データソースを集約して構築されるが、上流は必ずしも安定せず、データの消失や公開済みアドバイザリの適時的な書き換えが現実に行われている。本研究では、この不安定性を定量観測するため、OSS 脆弱性スキャナ Vuln の DB 公開 CI に公開前検査を実装した。検査は公開候補と直前の公開版を、検知結果と検知条件構造の 2 観点で比較し、変動率が閾値を超えた場合は公開を保留して人間の確認に委ねる。82 日間の本番運用の全数調査では 50 件の独立イベントで検査が発動し、全工程を履歴管理する既報の基盤を用いて、その全てについて原因が上流データの変化か、変換や DB 構築といった自前の処理かを切り分けて特定できた。日常変動と構造的イベントは変動率で桁が分離しており、閾値による単純な判定が実用になること、また欠損 DB の公開を 3 回阻止したことを示す。

キーワード：脆弱性データベース、データ品質、公開前検査、履歴管理、ソフトウェアサプライチェーン

Measuring Upstream Instability of Vulnerability Information: An 82-Day Exhaustive Study of Pre-Publication Inspection on a History-Managed Database Pipeline

SHUNICHI SHINOHARA^{1,a)} NORIHIRO NAKAOKA^{1,b)} KOTA KANBE^{1,c)}

Abstract: Vulnerability databases are built by aggregating many upstream data sources, but these upstreams are not always stable: data disappears, and published advisories are retroactively rewritten. To quantify this instability, we implemented a pre-publication inspection in the CI pipeline that builds and publishes the database of Vuln, an open-source vulnerability scanner. The inspection compares each publication candidate with the previously published version in terms of detection results and detection-criteria structure, and when the change rate exceeds a threshold, it withholds publication pending human review. In an exhaustive study covering 82 days of production operation, the inspection fired on 50 independent events, and our previously reported infrastructure, which keeps every pipeline stage under version control, allowed us to attribute every event to either upstream data changes or our own processing such as data transformation and database construction. We show that everyday fluctuation and structural events are separated by an order of magnitude in change rate, making this simple threshold-based inspection practical, and that the inspection blocked the publication of defective databases three times.

Keywords: Vulnerability Database, Data Quality, Pre-Publication Inspection, History Management, Software Supply Chain

1. はじめに

脆弱性対応の実務は、ディストリビュータやベンダが公開するアドバイザリや脆弱性フィードを「正」として組み立てられている。脆弱性スキャナはこれらを集約した脆弱

¹ フューチャー株式会社

Future Corporation

a) s.shinohara.yx@future.co.jp

b) n.nakaoka.46@future.co.jp

c) k.kanbe.r4@future.co.jp

性データベース (以下、脆弱性 DB) に依存し、検知結果はフィードの内容をほぼそのまま反映する。ここには、上流フィードは概ね安定しており、変化は新規脆弱性の追加という単調な形をとる、という暗黙の仮定が置かれている。この仮定はどの程度正しいのだろうか。

我々は OSS 脆弱性スキャナ Vulsc[1] のエコシステムにおいて、40 超のデータソースから脆弱性 DB(vuls.db) を CI で自動ビルドし、6 時間おきに公開している。前稿 [2] では、この収集 (fetch)、変換 (extract)、構築 (db build) の全工程をバージョン管理システムと連携させ、脆弱性情報の変更履歴を客観的事実として追跡・再現可能にする履歴管理基盤を報告した。本稿はその続編にあたる。前稿の基盤が「脆弱性情報の変化を記録できる」ことを示したのに対し、本稿はその記録能力を使って「上流は実際にどう変化しているか」を系統的に測定し、さらにその測定を DB の公開品質管理として実用化した結果を報告する。

測定の装置は単純である。公開直前の候補 DB を、現に公開されている直前版 (baseline) と自動比較し、変動率が閾値を超えたら公開を保留して人間に見せる。我々はこれを公開前検査 (diff guard) と呼ぶ。導入の直接の動機は、上流の破損データとパイプライン変更の複合、および DB とスキャナのバージョン非互換によって「壊れた DB」を公開してしまった 2 件のインシデント (2.2 節) であり、検査は第一義には安全ネットである。しかし 82 日間の本番運用で得られた発動記録の全数調査は、副産物として、脆弱性情報の上流がいかに大きく、頻繁に、時に黙って動くかの縦断観測データとなった。

本稿の貢献は次の 3 点である。

- (1) **上流不安定性の実測** (5 節, 6 節): 82 日間・671 run の全数調査により、上流フィードの不安定性を 50 件の独立イベントとして分類・定量化した。観測されたイベントは、可用性の不安定 (月次データの消失とフラッピング)、完全性の危うさ (遡及的な再キュレーションやアナウンスのないサーバ側再編)、スケールの分布 (日常変動と構造的イベントの析分離) の 3 群に整理できる。これらの上流は他の脆弱性ツール (Trivy, Grype, OSV 系等) も共有しており、知見はエコシステム横断で再利用可能である。
- (2) **原因帰属の方法論** (4 節): 発動の原因をビルダ、抽出器、上流の順に容疑を除外して確定する切り分け手順を定義し、全イベントに適用した。各ステップは履歴管理基盤 [2] の対応する工程の履歴 (DB メタデータ、抽出器のコミット履歴、raw データの Git 差分) に依拠しており、全工程の履歴管理が帰属の成立条件であることを示す。
- (3) **公開前検査の設計と運用機構** (3 節): 検知結果差分と DB 構造差分の 2 観点による検査本体に加え、監査証跡付き手動 override(promote)、対象別閾値の統計的

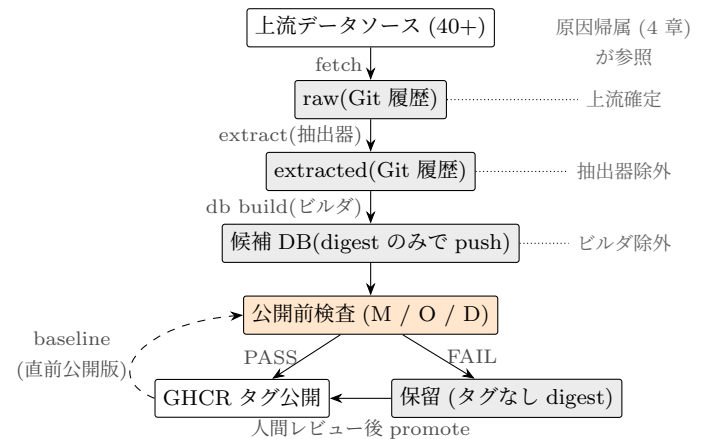


図 1 脆弱性 DB 公開パイプラインと公開前検査。全工程が Git 履歴管理 [2] され、原因帰属 (4 章) は各工程の履歴を参照する。

導出と定期的な再導出 (grooming) という、検査を運用し続けるための機構を設計・実装した。検査は欠損 DB の公開を 3 回阻止した。

2. 背景

2.1 履歴管理基盤 (前稿) の要点

vuls.db の生成は 3 工程からなる [2], [3], [4]。fetch 工程は一次情報源から取得したデータを原形をほぼ保った vuls-data-raw 形式で、extract 工程はそれを共通スキーマの vuls-data-extracted 形式で、それぞれ Git 管理下に置く。db build 工程は extracted データから BoltDB 形式の vuls.db を構築し、コンテナレジストリ (GHCR) にタグおよび digest で公開する。以下本稿では、extract 工程の変換コードを抽出器、db build 工程の構築コードをビルダと呼ぶ。この構成により、(1) 任意時点の上流データの状態をコミット単位で遡れる、(2) DB を digest で固定して検知結果を再現できる、という 2 つの性質が得られている。本稿の測定はこの 2 性質の直接の応用である。

2.2 動機となった 2 つのインシデント

2026 年 3 月、生成物としての DB が壊れているのに公開が成功してしまう事象を 2 件経験した。

インシデント 1(RHEL VEX, false negative): パイプラインにデータ形式変更を投入したのと同じ日に、Red Hat が壊れた VEX データを配信した。抽出処理はエラー終了してデータが旧形式のまま残り、新形式を前提とする後段フィルタが空データを生成した。結果、redhat:9 で本来検知されるべき CVE の 51.9%(3,387 件) が「影響なし」となる DB が公開された。

インシデント 2(Ubuntu vulnerable: false, false positive): DB 側に導入されたマーカーを、それを解釈するフィルタを持たない旧バージョンのスキャナが読んだ結果、ubuntu.2204 の検知件数が 5,968 件から 12,296 件 (+106.0%) に倍増した。DB 単体でもスキャナ単体でも正

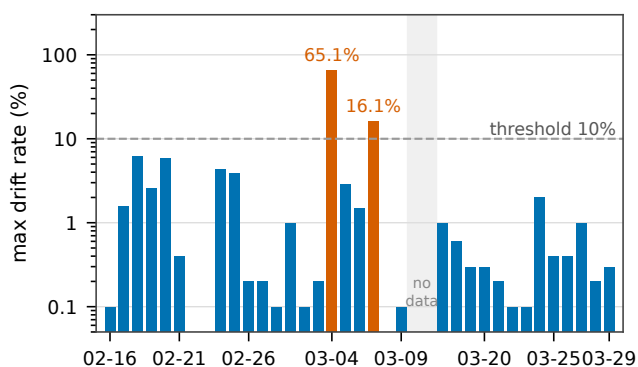


図 2 導入前ベンチマーク: 連続 1 日ペア 35 組の最大変動率 (対数軸). 破線は db 既定閾値 10%. 橙の 2 ペアが実在の上流イベント (Red Hat VEX 構造変更 65.1%, SUSE OVAL 再発行 16.1%) であり, 正常帯 (中央値 0.3%) と桁で分離する.

常であり, 組み合わせの非互換が原因である.

従来の CI が検査するのはビルドの成否であって, 生成物のデータとしての妥当性ではない. また 2 件は対照的で, 前者は DB 単体を見れば検出できるが, 後者は DB とスキャナ (の特定バージョン) の組で実行しなければ検出できない. この 2 面性が次章の設計に直結する.

3. 公開前検査の設計

3.1 導入前ベンチマーク: 閾値検査の成立可能性

脆弱性 DB は日々更新されるため差分ゼロはあり得ず, 閾値による検査が成立するには正常な変動と異常なイベントが変動率で分離する必要がある. 導入前に, GHCR 上に履歴として残っている過去 37 日分の nightly DB(2026-02-15 から 03-29) を用い, 連続 1 日ペア 35 組で DB 構造差分を実測した. 通常運用の変動率は中央値 0.3%, 最大 6.2% に収まり, 期間中に 10% を超えた 2 組はいずれも実在の上流データ品質イベント (Red Hat VEX 構造変更で redhat:6 が 65.1%, SUSE OVAL 大規模更新で 12 から 16%) であった. 正常帯と異常イベントが桁で分離するというこの観測が, 閾値検査成立の根拠である. なお, 過去任意時点の DB ペアでこの種のベンチマークが実行できること自体, 公開物が digest 付きで履歴化されている [2] ことの恩恵である.

3.2 3 つのチェック

公開 CI の圧縮後・公開前に, 次の 3 チェックを実行する (以下 M / O / D と略記).

- **M: 検知結果差分 (スキャナ最新版).** baseline と target の両 DB に対して同一のスキャン結果 fixture 群 (実機スキャン由来, 数十ファイル) でスキャナを実際に実行し, ファイル単位で検知 CVE 集合を比較する. 変動率は (追加数+削除数)/baseline 検知数で定義する.
- **O: 検知結果差分 (固定した旧バージョンのスキャナ).** 過去バージョンとの組でのみ現れる非互換退行 (イン

シデント 2 型) を捕捉する.

- **D: DB 構造差分.** BoltDB を直接開き, ecosystem(後にソース単位へ細分化) ごとに検知条件木を leaf Criterion レベルまで平坦化して構造比較する. スキャン結果もスキャナも不要で DB のみで完結する. 比較粒度をバイト列ではなく Criterion にしたのは, JSON キー順などの無害な変更での偽陽性を避けつつ, 分母 (全体で約 330 万 Criterion) を大きく取り正常変動を小さな drift として吸収するためである.

チェック結果は変動率, 増減の内訳, 変動した CVE やアドバイザリの ID を含む構造化されたレポートとして常に CI の Job Summary に出力される. これは前稿が課題として挙げた「行ベース差分ではなく論理的な構造としての差分提示」[2] の, 公開品質管理という文脈での実装形でもある. 導入当初は最初の FAIL で検査が停止する実装だったが, 先に FAIL したチェックが後段のシグナルを隠す問題が判明し (6.4 節), 現在は 3 チェックすべてを実行してから集約判定する.

3.3 FAIL 時の運用: 監査証拠付き手動 promote

候補 DB は検査実行前に digest のみ (タグなし) で GHCR に push しておき, PASS 時にのみタグを付けて「公開」する. FAIL した場合, 候補はタグなしのまま digest で取得可能な状態で残るため, 事後検証が可能である. オペレータがレポートを確認し正当な変更と判断した場合は, 専用 workflow で該当 digest に手動でタグ付けする (promote). 実行者, digest, タグが run 履歴に残り, 「誰が何を根拠に override したか」が追跡できる. 運用ルールとして, promote の判断・実行は人間に限定している (調査を支援する AI エージェントには実行させない).

3.4 閾値の per-target / per-source 化と grooming

閾値はデフォルトで detection 5% / db 10% とし, 運用実績に基づき対象別に緩和する override 機構を持つ. 導入 1 ヶ月の運用で, 新ディストリビューション立ち上がり期の一括 triage, rolling release, ベンダ月次サイクルといった反復的な正当変動が毎回検査に引っかかることが判明したためである. override 値は失敗履歴の統計から「観測ピーク+ヘッドルーム」で導出し, 単発スパイクは override せず FAIL させて人間に見せる (例: EOL ディストリビューションが 39% 動くのは異常として意図的に残置). リストの陳腐化を防ぐため約 2 ヶ月周期で全再導出する (grooming). さらに運用後期, ecosystem 一括の変動率では大きなソースがノイズフロアになって小さなソースの異常を希釈する問題が顕在化し (6.3 節), 閾値もレポートもソース単位に細分化した.

4. 測定方法論: 全数調査と原因帰属

全数調査: 本番導入 (2026-04-23) から 07-14 までの 82 日間, 2 系統 (安定版 / nightly) の全 671 run を対象に, run ログ, PR, promote 履歴を GitHub API で全数収集した. FAIL 中は promote があるまで baseline が動かず, 同じ差分で 6 時間おきの cron が再 FAIL し続けるため, run 数は重複を含む. そこで連続 FAIL を 1 つに潰した fail sequence(66 件), さらに 2 系統を束ねソース単位の出現時系列で分解した source-episode(50 件) を独立イベントの単位として定義した.

原因帰属: 各イベントについて, 次の順で容疑を除外して原因を確定する.

- (1) **ビルダ除外:** baseline と target の両 DB のメタデータに記録されたビルダのバージョンを比較する. 一致すれば DB 構築コードは無罪.
- (2) **抽出器除外:** 対象期間の抽出器リポジトリのコミット履歴を走査する. 当該ソースの fetch / extract コードに変更がなければ変換コードは無罪.
- (3) **上流確定:** raw データの Git 履歴から対象期間のコミットを特定し, 差分の中に変動と件数・ID が一致する変更実体 (smoking gun. 例えば新規アドバイザリファイル群やディレクトリの消失) を突き止める.

この手順の各ステップは, 履歴管理基盤 [2] の対応する工程の履歴にそれぞれ依拠している. ビルダ除外は DB メタデータ, 抽出器除外はコードのコミット履歴, 上流確定は raw および extracted データの Git 履歴である. 全工程が履歴化されていなければ, 例えば「上流の一時障害なので候補を破棄すべき」と「正当な変更なので promote してよい」の区別 (5.2 節) は当て推量になる. 逆に言えば, 履歴管理は本測定の成立条件であり, 前稿の基盤の実証でもある. 実際, 全 50 イベントで帰属は完了し, 「原因不明」は残らなかった. また帰属手順は再現可能な triage 手順書として整備し, チームで共有している.

5. 運用結果

5.1 規模と原因分布

671 run 中, 失敗は 339 run. うち 295 run が検査のステップで失敗した (閾値超過 293+検査内インフラ障害 2). 独立イベントは 50 source-episode, 事例カタログとしては約 21 イベント (複数ソースの同時多発を 1 事例に束ねた粒度) である. 原因の分布を表 1 に示す.

5.2 実害を防いだ 3 事例

- (1) **Microsoft CVRF 月次データ全消失 (2 回):** 上流 API が当月分のセキュリティ更新ドキュメント (700 から 1,300 ファイル) を丸ごと返さなくなり, Windows

表 1 原因帰属の分布 (事例単位)

判定	件数	代表例
上流由来 (a) 正当な変更	約 15/21	月次パッチ等
上流由来 (b) 一時的障害 (公開阻止)	2	CVRF 消失 ×2
上流由来 (c) 恒久的品質イベント	2	errata 再編
抽出器由来・コードバグ (公開阻止)	1	非決定的出力
抽出器由来・意図的変更	1	CPE 分類導入
ビルダ由来	0	(2 件精査, 無罪)

系 13 ターゲットで最大 53.6% の削除のみの差分が発生した. 公開されていればインシデント 1 と同型の大量 false negative である. 検査は候補を破棄させ, 上流回復後の再実行のみで復旧した. raw 履歴上, 当月ファイル数は 1273 → 0 → 1273 → 0 → 1278 とフラッピングしており, 単発の取得失敗ではなく上流配信自体の不安定であった (フラップの各時点がコミットとして残っているため, この認定自体が履歴管理の産物である).

- (2) **抽出器の非決定性バグ:** raw の変更は 266 ファイルなのに extracted は 23,163 ファイル動くという異常を D チェックが 274.5% の drift として検出した. 原因は抽出器が型の取り違えでソート処理をスキップし出力順序が非決定になっていたことで, 修正 PR に帰結した. 期間中唯一の「上流のせいではないコード起因の異常」の捕捉である.
- (3) **(限定的)VulnCheck 一斉ロールアウト:** 単一コミットで 145,672 ファイル, +957 万行という生成 CPE 設定の一斉付与を D チェックが検出した. ただし当時の ecosystem 一括の変動率では「cpe 28.9%」としか言えず, 候補は promote された. 事後のソース単位の再分析で, 当該ソース単独では 189.6% の変動であり, かつ実検知が 60 から 75% 消失する退行を含んでいたことが判明した. この教訓がソース単位への細分化 (3.4 節) の直接の動機である. 「発動したが発解能不足で判定を誤りかけた」事例として記録する.

5.3 false positive 論と運用ループ

発動の大半は正当な上流変更であり, これを「誤報」と見るかは設計思想による. 本検査は安全ネットであり, 822 CVE を単一アドバイザリに同梱するカーネル更新や, 約 600 パッケージを束ねた単一アドバイザリ (5,312 検知条件) のマス更新は, 正当であっても公開前に人間が一目見るに値する. 一方, 反復性が確立した既知の変動は override で恒久緩和する. 82 日間の手動 promote は 59 unique digest あり, その 100% が検査 FAIL run の候補 digest と一致した. すなわち, 検査, 人間によるレポート確認, 監査証跡付き override というループが例外なく回っていたことを示す.

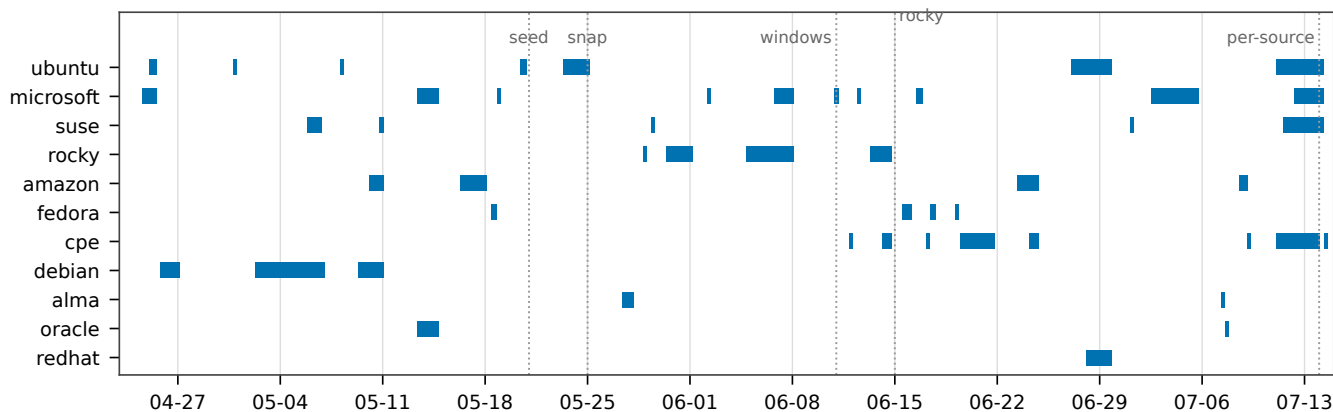


図 3 82 日間の発動タイムライン (source-episode 50 件). 横棒は各 episode の持続期間 (FAIL 初出から終端まで), 点線は override 導入時点 (seed: 初期シード, snap: ubuntu:snap, windows: windows 系 detection, rocky: rocky_10, per-source: ソース単位化). per-source 化後の cpe.*系は cpe 行に含めた.

6. 考察: 観測された上流の危うさ

6.1 消える・戻る・また消える (可用性)

月次データの丸ごと消失 (5.2 節) は 2 ヶ月連続で発生し, 2 回目は 2 日間に消失と復活を 4 回繰り返した. フィードの取得は「ある時点のスナップショット」であり, 欠損の瞬間を取り込めばビルドは成功したまま欠損 DB ができあがる. 公開前の差分検査なしにこれを防ぐ手段は事実上ない.

6.2 遡及的に書き換わる (完全性・一貫性)

- **errata の再キュレーション:** AlmaLinux の errata フィードが 1 週間に複数回再編成され, アドバイザリ数が 279 から一時 60 まで減少, 削除されたアドバイザリが参照する CVE の 87%(376 件) がフィードから完全消失した. 生き残ったアドバイザリも, kernel 更新の検知条件が 73 から 2 パッケージへ縮退する「骨抜き」を伴った. 過去にも同様のページ実績がある反復性の高い上流運用である. 対応として, 劣化ソースの取り込みをコミット単位でピン留めし, 代替ソース (OVAL) の抽出器を整備してから計画的にアンピンした. ピン留めという対応自体, 取り込みが履歴管理されているから可能な操作である.
- **サイレントなサーバ側再編:** Cisco openVuln API では, アドバイザリの更新日時もバージョンも一切変わらないまま, アドバイザリと影響バージョンのマッピングだけが日次で書き換わり, 93 アドバイザリから製品バージョン 2,022 件が削除された (例: 2014 年のアドバイザリの製品リストが 149 から 36 へ). changelog にも issue にもアナウンスはない. 過剰マッピングの整理というデータ品質改善の側面が強い一方, 該当バージョンを監視している利用者には黙った検知消失

となる.

- **一斉再生成の両義性:** VulnCheck の全年代 CVE への生成 CPE 一括付与 (5.2 節) は情報の拡充である一方, CPE 語彙の現代化により既存のマッチングから外れる検知退行を同時に含んでいた. 2 週間後, 上流側の修正により退行の一部は「回復イベント」としてもう一度検査を発動させた. 拡充, 退行, 回復のいずれもが大変動として観測される.

6.3 正当でも桁違いに動く (スケール)

観測された最大変動率は 428.3%(AlmaLinux 再キュレーション), 305.9%(CPE 系), 274.5%(抽出器バグ) に達する一方, 日常変動は 5 から 20% 帯に収まり, 両者は概ね桁で分離する. これは導入前ベンチマーク (3.1 節) の観測が本番でも維持されたことを意味し, 単純な閾値検査が実用になる理由である. ただし分離が崩れる構造要因が 2 つある. (1) baseline が小さい対象は 1 バッチの正当な追加で容易に 100% を超える. (2) FAIL で baseline が停滞すると比較窓が伸び, 日常変動が蓄積して閾値を超える. 前者は per-target 閾値, 後者は迅速な promote 運用で吸収している.

6.4 検査自身の盲点と改良

運用は検査自身の弱点も露呈させた. (1) Windows 更新プログラム (KB) の比較はキー集合しか見ておらず, CVE エントリの大量消失時に 0% を返した (「0% ≠ 変化なし」). (2) 導入初期は最初の FAIL で停止する実装で, 先に FAIL したチェックが別の異常を隠した (全チェック実行と集約への変更で対処). (3) ecosystem 一括の変動率は大ソースが小ソースの異常を希釈した (ソース単位への細分化で対処). (4) override の系統間非対称が片系統だけの慢性 FAIL を生んだ (grooming で両系統を同時に見ることで対処).

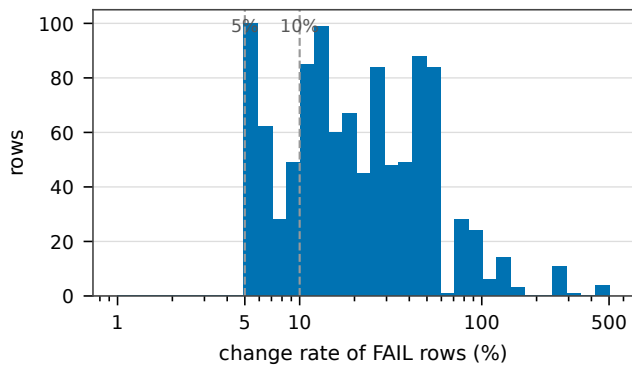


図 4 全 FAIL 行 (1,040 行) の変動率分布 (対数軸). 破線は既定閾値 (detection 5% / db 10%). 日常変動の本体に対し, 100% 超の構造イベント群が裾として分離する. FAIL 中は cron 再実行により同一イベントの行が繰り返し計上される点に注意.

検査は一度作って終わりではなく, 発動事例からのフィードバックで分解能と運用を継続的に改良する対象である.

6.5 運用コストの実態

run 単位の FAIL の 52% は土日に発生しているが, これは上流ではなく人間側の信号である. promote が平日日中にしか行われないため, 金曜夕方以降のイベントが週末じゅう再 FAIL し続ける (週末滞留). fail sequence の持続時間 (中央値 11h / p90 48h) は検査の検出性能ではなくオペレータの応答時間の分布を測っており, 通知や承認フローの自動化余地を見積もる直接の根拠となる.

6.6 開発プロセスに対する最終ガードレールとしての価値

本検査の副次的だが実感の大きい価値として, パイプライン開発の最終防衛線としての機能がある. ユニットテストやコードレビューが工程ごとのコードの正しさを検査するのに対し, データパイプラインの意味的な正しさは集約後の出力でしか観測できない. 実際, 型の取り違いによるソート処理の欠落 (5.2 節) はコンパイルもテストも通過する類のバグであり, 出力の構造差分としてのみ顕在化した. また運用上は, 実験系統 (nightly) に破壊的変更を先行投入し検査で影響規模を定量観測してから安定版へ降ろす, 変更前から FAIL を予期しその発現範囲が想定に閉じていることを混入なしの証拠として使う (6.2 節のピン解除), といった能動的な使い方も定着した. この「作られ方に依存せず生成物を検証する」性質は, AI エージェントによるコード生成が開発の速度と量を押し上げる今後, コードレビューのスケール限界を補う出力レベルのガードレールとして重要性を増すと考える. なお本運用では, 検査を通す判断 (promote) は人間に限定し, AI エージェントには調査支援までを許している (3.3 節). 生成は速く, 検証は出力で, 承認は人間で, という役割分担である.

7. 関連研究

脆弱性データベースの品質: NVD をはじめとする脆弱性 DB の品質は繰り返し測定されてきた. CVE/NVD と外部レポート間の影響バージョン不整合の大規模検出 [5], NVD, JVNDB, CNVD の登録プロセスとカバレッジの比較 [6], NVD コーパスの品質監査と自動修正 [7], 影響バージョン情報の実地検証 [8], CVSS 付与遅延の回帰分析 [9], NVD 利用者の定性調査 [10] などがある. また, 同一対象に対するスキャナや SCA ツール間の検知結果の大きな不一致が, 参照する脆弱性 DB の差に相当程度帰着することも示されている [11], [12]. 2024 年の NVD enrichment 停滞は産業側の分析でも定量化された [13], [14]. これらはいずれもフィードを外部から静的に (スナップショット間比較, コーパス監査, 回顧的分析として) 測定するものである. 本稿はこれに対し, 測定器を本番の取り込み・公開パイプラインに常設し, フィード内容の経時変化そのものを下流消費者の視点で縦断観測した点で異なる.

セキュリティデータフィードの品質測定: 脅威インテリジェンスフィードについては, 量, カバレッジ, 遅延, 正確性等の指標による比較測定が行われ, フィード間の重複の少なさや, 消費者にとって品質が不可視であることが示されている [15], [16], [17]. フィードの内容品質を測るという点で方法的に本稿へ最も近い系譜だが, いずれも外部の研究として実施された横断測定である. 本稿は, 単一フィードを自身の公開履歴と比較する縦断測定を, 公開を保留する運用機構として常設した点が新しい.

ソフトウェアサプライチェーン保全: SLSA[18], in-toto[19], reproducible builds[20], Sigstore[21], TUF[22] は, artifact の来歴, ビルド完全性, 署名, 配布を保護する枠組みである. これらは「正規の上流から正規の手順で作られ, 改竄されていないこと」を保証するが, 正規に取得され正しく署名された DB の内容が, 上流の劣化により意味的に欠損していても全チェックを通過する. 本稿の検査は, これらの枠組みが守る来歴・完全性レイヤの上に, データ内容の意味レイヤの検証を積む補完関係にある.

データパイプラインの検証: 宣言的制約によるデータ品質検証 [23], [24] に加え, パイプライン自身の実行履歴から検証基準を自動導出する研究 [25], [26], 新しいデータバッチが履歴から逸脱した際に下流の ML 再学習をブロックするゲート [27] があり, 特に後者は「履歴との比較で公開・利用を止める」という意味論まで本稿と同型である. テスト理論の観点では, 本稿の検査は前公開版を比較対象 (オラクル) とする differential testing[28] の時間軸変種, あるいはデータ artifact に対する回帰テスト [29] の適用とみなせる. これらとの差分は, (1) 列統計ではなく検知結果と検知条件構造という意味レベルの差分を取ること, (2) セ

セキュリティクリティカルな公開物を対象に、監査証跡付きの human-in-the-loop override を備えること、(3) 検査の発動記録そのものを上流不安定性の測定データとして全数分析したこと、の3点である。

国内の関連研究: 国内では、脆弱性対策情報データベース JVN の提案 [30] に始まる脆弱性情報流通の系譜があり、近年は本トラックで OSS の脆弱性修正状況の収集と調査 [31]、PSIRT 向け脆弱性スクリーニング [32] など、公開脆弱性情報とその利用の間の乖離を扱う研究が続いている。また IPA の JVN iPedia 登録状況レポート [33] は、2024 年の NVD 公開遅延が国内 DB の登録数を減少させたことを公式に記録しており、上流の不安定性が国内の脆弱性情報基盤へ波及する実例である。一方、フィード内容の経時的不安定性を定量計測した国内の学術研究は、我々の調査の範囲では見当たらない。

8. おわりに

脆弱性 DB の公開 CI に公開前検査を実装し、82 日間・671 run の全数調査を通じて、脆弱性情報の上流ソースの不安定性を 50 件の独立イベントとして定量観測した。上流は消え、戻り、遡及的に書き換わり、アナウンスなく再編される。しかもその大半は正当な変更である。得られた知見は次の通りである。(1) 正常変動と異常イベントは変動率で概ね桁分離し、単純な閾値検査が実用になる。ただし分解能 (ソース単位) と閾値 (対象単位の override と定期 grooming) は運用実績から継続的に導出する必要がある。(2) 発動原因の帰属は、収集・変換・構築の全工程が履歴管理されていることを成立条件とする。前稿で報告した履歴管理基盤 [2] は、本測定と公開品質管理の土台として実証された。本稿は同稿が課題として挙げた「差分の論理的な提示」「パイプラインの堅牢化」への、運用を伴う一つの回答でもある。(3) FAIL 時の復旧は「人間によるレポート確認+監査証跡付き override」として設計すべきで、本運用では全 override がこのループに乗った。(4) 上流の恒久的劣化には、取り込みのピン留めと代替ソース整備という出口が要る。(5) 生成物を出力レベルで検証する検査は、上流だけでなく自らの開発プロセスに対する最終防衛線としても機能し、テストとレビューをすり抜ける意味的バグの捕捉や、破壊の変更の影響範囲の定量確認に実際に寄与した。コード生成に AI が入り込む開発形態において、この価値は今後増すと考える。

今後の課題として、fetch 段階での欠損検出 (月次ドキュメント丸ごと消失時にコミットしない等)、通知や承認フローの改善による週末滞留の解消、観測の長期蓄積による時系列特性 (月次サイクル等) の統計的確立、および同一上流を消費する他エコシステムとの観測の突合が挙げられる。

参考文献

- [1] Vulns: Vulnerability scanner. <https://github.com/future-architect/vuls>. (Accessed 2026-07-29).
- [2] 中岡典弘, 篠原俊一. 広範な脆弱性情報の統合管理と履歴追跡. コンピュータセキュリティシンポジウム 2025 (CSS 2025), 2025.
- [3] vuls-data-update. <https://github.com/MaineK00n/vuls-data-update>. (Accessed 2026-07-29).
- [4] vuls-data-db. <https://github.com/vulsio/vuls-data-db>. (Accessed 2026-07-29).
- [5] Ying Dong, Wenbo Guo, Yueqi Chen, Xinyu Xing, Yuqing Zhang, and Gang Wang. Towards the detection of inconsistencies in public security vulnerability reports. In *28th USENIX Security Symposium*, pp. 869–885, 2019.
- [6] Juehao Lin, Xuanxiang William Wang, Gianluca Stringhini, and Manuel Egele. A comparative analysis of nvd, jvndb and cnvd: Insights into global and regional vulnerability reporting. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2026.
- [7] Afsah Anwar, Ahmed Abusnaina, Songqing Chen, Frank Li, and David Mohaisen. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. *IEEE Transactions on Dependable and Secure Computing*, Vol. 19, No. 6, pp. 4255–4269, 2022.
- [8] Viet Hung Nguyen and Fabio Massacci. The (un)reliability of NVD vulnerable versions data: An empirical experiment on google chrome vulnerabilities. In *8th ACM SIGSAC Symposium on Information, Computer and Communications Security (ASIA CCS)*, pp. 493–498, 2013.
- [9] Jukka Ruohonen. A look at the time delays in CVSS vulnerability scoring. *Applied Computing and Informatics*, Vol. 15, No. 2, 2019.
- [10] Julia Wunder, Alan Corona, Andreas Hammer, and Zinaida Benenson. On NVD users' attitudes, experiences, hopes, and hurdles. *Digital Threats: Research and Practice*, Vol. 5, , 2024.
- [11] Nasif Intiaz, Seaver Thorn, and Laurie Williams. A comparative study of vulnerability reporting by software composition analysis tools. In *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2021.
- [12] Yekaterina Churakova, Mathias Ekstedt, and Larissa Schmid. Vexed by VEX tools: Consistency evaluation of container vulnerability scanners. arXiv:2503.14388, 2025.
- [13] VulnCheck. The real danger lurking in the NVD backlog. <https://www.vulncheck.com/blog/nvd-backlog-exploitation>, 2024. (Accessed 2026-07-29).
- [14] Anchore. The NVD enrichment crisis: One year later. <https://anchore.com/blog/nvd-crisis-one-year-later/>, 2025. (Accessed 2026-07-29).
- [15] Vector Guo Li, Matthew Dunn, Paul Pearce, Damon McCoy, Geoffrey M. Voelker, Stefan Savage, and Kirill Levchenko. Reading the tea leaves: A comparative analysis of threat intelligence. In *28th USENIX Security Symposium*, pp. 851–867, 2019.
- [16] Xander Bouwman, Harm Griffioen, Jelle Egbers, Christian Doerr, Bram Klievink, and Michel van Eeten. A different cup of TI? the added value of commercial threat intelligence. In *29th USENIX Security Symposium*, 2020.
- [17] Harm Griffioen, Tim Booij, and Christian Doerr. Quality

evaluation of cyber threat intelligence feeds. In *Applied Cryptography and Network Security (ACNS), LNCS 12147*, pp. 277–296, 2020.

- [18] OpenSSF. SLSA: Supply-chain levels for software artifacts. <https://slsa.dev>. (Accessed 2026-07-29).
- [19] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pp. 1393–1410, 2019.
- [20] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, Vol. 39, No. 2, pp. 62–70, 2022.
- [21] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
- [22] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *17th ACM Conference on Computer and Communications Security (CCS)*, pp. 61–72, 2010.
- [23] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. Automating large-scale data quality verification. *Proceedings of the VLDB Endowment*, Vol. 11, No. 12, pp. 1781–1794, 2018.
- [24] Eric Breck, Marty Zinkevich, Neoklis Polyzotis, Steven Euijong Whang, and Sudip Roy. Data validation for machine learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.
- [25] Dezhan Tu, Yeye He, Weiwei Cui, Song Ge, Haidong Zhang, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines. In *29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023.
- [26] Sergey Redyuk, Zoi Kaoudi, Volker Markl, and Sebastian Schelter. Automating data quality validation for dynamic data ingestion. In *24th International Conference on Extending Database Technology (EDBT)*, pp. 61–72, 2021.
- [27] Shreya Shankar, Labib Fawaz, Karl Gyllstrom, and Aditya G. Parameswaran. Moving fast with broken data. arXiv:2303.06094, 2023.
- [28] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, Vol. 10, No. 1, pp. 100–107, 1998.
- [29] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification and Reliability*, Vol. 22, No. 2, pp. 67–120, 2012.
- [30] 寺田真敏, 高田真吾, 土居範久. 脆弱性対策情報データベース JVN の提案. 情報処理学会論文誌, Vol. 46, No. 5, pp. 1256–1265, 2005.
- [31] 葛野弘樹, 矢野智彦, 面和毅, 山内利宏. オープンソースソフトウェアを対象とした脆弱性修正状況の収集と調査. コンピュータセキュリティシンポジウム 2024 (CSS 2024), 2024.
- [32] 金井遵, 上原龍也, 小池竜一, 鬼頭利之, 神戸康多, 木戸俊輔, 篠原俊一, 中岡典弘, 林優二郎. PSIRT 向け脆弱性スクリーニング技術と環境毎リスク評価技術. コンピュータセキュリティシンポジウム 2024 (CSS 2024), 2024.
- [33] 情報処理推進機構. 脆弱性対策情報データベース JVN iPedia の登録状況. [https://www.ipa.go.jp/security/](https://www.ipa.go.jp/security/reports/vuln/jvn/index.html)

[reports/vuln/jvn/index.html](https://www.ipa.go.jp/security/reports/vuln/jvn/index.html). 四半期レポート. (Accessed 2026-07-29).

9. 研究倫理

本研究の観測対象は、一般に公開されている脆弱性情報フィードと、我々自身が運用する公開 CI の実行記録のみであり、脆弱性の新規発見、実システムへの攻撃、個人情報の取り扱いを伴わない。したがって本研究は倫理的な問題に関係しない。なお、本稿で報告する上流のデータ品質イベントは公開データに対する機械的な観測から得られた事実であり、特定のベンダや組織の評価・非難を意図するものではない。